

Club de Programación Avanzada UAM Iztapalapa

NOTAS PARA UNA INTRODUCCIÓN A LA PROGRAMACIÓN COMPETITIVA EN C++

Ing. Luis Daniel Ramos López

Última actualización:

22-08-2026

Índice

1. Cabeceras e inicio del programa	3
1.1. Biblioteca <code><bits/stdc++.h></code>	3
1.2. Uso de <code>using namespace std</code>	3
1.3. Desincronización de <code>iostream</code> con <code>stdio.h</code>	3
1.4. Compilación y ejecución	3
2. Algunas bases de programación estructurada	4
2.1. Leer y escribir variables	4
2.2. Estructuras de condición y repetición	4
2.3. Declaración de arreglos	5
2.4. Tipos de datos	6
2.5. Tipos de datos flotantes	6
2.6. Operaciones con bits	7
3. Algoritmia y eficiencia	8
3.1. Límite en tiempo	8
3.2. Límite en memoria	9
4. Problemas clásicos de algoritmia sobre arreglos	9
4.1. Ocurrencias en una lista	9
4.2. Sumas de prefijos en una lista	10
4.3. Problema de la mayor suma de un subarreglo de tamaño k	11
5. Apuntadores, la biblioteca <code>algorithm</code> y estructuras	13
5.1. Apuntadores	13
5.2. Algunas funciones básicas de la biblioteca <code>algorithm</code>	13
5.3. Paso por copia y por referencia	14
5.3.1. Paso por copia	14
5.3.2. Paso por referencia	14
5.3.3. Referencia constante	14
5.4. Recursión	15
5.5. Pares (<code>pair</code>)	16
5.6. Declaración de estructuras	16
6. Algoritmos de ordenamiento y de búsqueda de secuencias	17
6.1. Ordenamiento de secuencias	17
6.2. Búsqueda eficiente sobre secuencias ordenadas	17
7. Estructuras de datos lineales	18
7.1. Cadenas de caracteres (<code>string</code>)	18
7.2. Arreglos dinámicos (<code>vector</code>)	20
7.3. Pilas (<code>stack</code>)	20
7.4. Colas (<code>queue</code>)	21
7.5. Dobles colas (<code>deque</code>)	21
7.6. Listas enlazadas (<code>list</code>)	22
7.7. Ciclo <code>for-each</code>	22
8. Estructuras de datos no lineales	23
8.1. Conjuntos (<code>set</code>)	23
8.2. Diccionarios (<code>map</code>)	24
8.3. Colas de prioridad (<code>priority_queue</code>)	24
8.4. Gráficas	25
8.4.1. Lista de adyacencia	25

8.4.2.	Matriz de adyacencia	26
8.4.3.	Comparación de representaciones	27
8.4.4.	Recorrido en amplitud (BFS)	27
8.4.5.	Recorrido en profundidad (DFS)	28
8.4.6.	Gráficas no conexas	28
8.4.7.	Comparación entre BFS y DFS	29
9.	Algunos algoritmos aritméticos básicos	29
9.1.	Operaciones con módulo	29
9.2.	Criba de Eratóstenes	30
9.3.	Coeficiente binomial iterativo	30
9.4.	Potencia rápida con módulo	31
9.5.	Binomial iterativo con módulo	31
9.6.	Factorización básica	32
9.7.	Máximo común divisor y mínimo común múltiplo	33
10.	Estructuras de datos avanzadas	33
10.1.	Árbol de segmentos	33
10.2.	Conjuntos disjuntos (DSU)	35
11.	Algunas consideraciones extras	36
11.1.	Uso de <code>auto</code>	36
11.2.	¿Qué hacer cuando voy a resolver un problema?	37

1. Cabeceras e inicio del programa

1.1. Biblioteca <bits/stdc++.h>

Para evitar poner cada biblioteca necesaria que se va a ocupar en un programa, se puede utilizar una biblioteca de C++ que **solo funciona con el compilador GCC** que permite utilizar cualquier función u objeto de la biblioteca estándar de C++ sin tener que ponerlas por separado.

```
#include <bits/stdc++.h>
```

Sin bits/stdc++.h:

```
#include <iostream>
#include <string>

int main() {
    std::string s; //Para usar cadenas usamos la biblioteca string
    cin >> s;      //Para entrada y salida estandar usamos la biblioteca iostream
    cout << s;
}
```

Con bits/stdc++.h

```
#include <bits/stdc++.h>

int main() {
    std::string s;
    std::cin >> s; //Aqui solo usamos un "include" en vez de dos
    std::cout << s;
}
```

1.2. Uso de using namespace std

Para evitar usar el prefijo “std::” para cada objeto o función de la biblioteca estándar de C++, usamos la directiva using namespace std.

```
#include <iostream>

int main() {
    std::cout << "Hola mundo";
}
```

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola mundo";
}
```

1.3. Desincronización de iostream con stdio.h

Para evitar la sincronización de iostream con stdio.h, lo cual es importante hacer cuando hay muchos elementos que leer o escribir solo con iostream, se usan dos líneas:

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr), cout.tie(nullptr);
    //... lo que sigue de código
}
```

1.4. Compilación y ejecución

Para compilar un archivo con extensión .cpp en la terminal, debemos estar posicionados en la carpeta que contiene el archivo y compilarlo de la siguiente manera:

```
g++ archivo.cpp -o archivo
```

Es importante saber que en un ambiente Linux los ejecutables, en general, no tienen extensión, a diferencia del .exe de los ejecutables en Windows. Para ejecutar en un ambiente de Linux, lo hacemos de la siguiente manera:

```
./archivo
```

2. Algunas bases de programación estructurada

2.1. Leer y escribir variables

Para leer y escribir variables desde la entrada estándar de cualquier tipo se utiliza `cin` (para leer) y `cout` (para escribir) de la biblioteca `iostream`:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    char c;
    string s;
    float f;
    cin >> n >> c >> s >> f;
    cout << "entero: " << n << " flotante: " << f << "\n";
    cout << c << " " << s << "\n";
}
```

Entrada:

```
5
p
gatito
3.14
```

Salida:

```
entero: 5  flotante: 3.14
p gatito
```

Nota: Evitar utilizar `endl` para salto de línea, en su lugar usen `"\n"`.

2.2. Estructuras de condición y repetición

Se pueden usar una condición `if` solamente:

```
if(condicion) {
    //codigo
}
```

También la puedes concatenar con un `else`:

```
if(condicion) {
    //codigo
} else {
    //codigo
}
```

Aunque también puedes concatenar varias condicionales:

```

if(condicion) {
    //codigo
} else if(condicion) {
    //codigo
} else {
    //codigo
}

```

También se puede usar la estructura **switch**:

```

switch(var) {
case 1:
    //codigo
    break;
case 2:
    //codigo
    break;
...
default:
    //codigo
    break;
}

```

Como estructuras de repetición tenemos la estructura **while**:

```

while(condicion) {
    //codigo
}

```

La estructura **do-while**:

```

do {
    //codigo
} while(condicion);

```

Y la estructura **for**, que, en este ejemplo, imprime los números del 0 al 2:

```

// var.  cond.  inc.
for(int i = 0; i < 3; ++i) {
    cout << i << "\n";
}

```

Salida:

```

0
1
2

```

2.3. Declaración de arreglos

Para declarar un arreglo nosotros usamos la sintaxis **tipo nombre[tam]**, donde **tipo** es el tipo de dato de nuestros elementos, **nombre** es el nombre de nuestro arreglo y **tam** es un entero que determina el tamaño del arreglo.

```

int arr[5];    //arreglo de cinco enteros

```

También podemos usar una variable de tipo entera para declarar el tamaño del arreglo:

```

int n = 3;
string nom[n];    //arreglo de cadenas de 3 elementos

```

Para acceder a un elemento del arreglo, para leerlo, modificarlo o imprimirlo, se usan los corchetes **[]**.

```

cin >> arr[3];    //leo un valor para el elemento 3 del arreglo
arr[1] = 5;        //le asigno un 5 al elemento 1 del arreglo
cout << arr[i];    //imprimo el i-esimo elemento del arreglo

```

Al declarar un arreglo, las direcciones de memoria traen por defecto basura, no es correcto suponer que tendrán cero en sus elementos inicialmente. Para declarar un arreglo de enteros con puros ceros de inicio se hace de este modo:

```
int arr[1000] = {};    //arreglo de 1000 elementos declarado inicialmente con ceros
```

2.4. Tipos de datos

A continuación, se muestra una tabla con los tamaños de los tipos de datos principales en C++, para saber si es conveniente usar un tipo de dato o puede desbordarse.

Tipo	Descripción	Tamaño	Rango de valores
bool	Valor booleano	1 byte	<i>true</i> o <i>false</i>
short	Entero corto con signo	2 bytes	$-3,2 \times 10^4$ a $3,2 \times 10^4$
unsigned short	Entero corto sin signo	2 bytes	0 a $6,5 \times 10^4$
int	Entero con signo	4 bytes	$-2,1 \times 10^9$ a $2,1 \times 10^9$
unsigned int	Entero sin signo	4 bytes	0 a $4,3 \times 10^9$
long long / int64_t	Entero largo con signo	8 bytes	$-9,2 \times 10^{18}$ a $9,2 \times 10^{18}$
unsigned long long / uint64_t	Entero largo sin signo	8 bytes	0 a $1,8 \times 10^{19}$
float	Punto flotante	4 bytes	$\pm 3,4 \times 10^{38}$ (7 dígitos de precisión)
double	Doble precisión	8 bytes	$\pm 1,7 \times 10^{308}$ (15 dígitos)
long double	Doble precisión extendida	16 bytes*	$\approx \pm 10^{4932}$ (mayor precisión)

*Nota: el tamaño de long double puede variar según el compilador, a veces 12, 16 o incluso 8 bytes.

2.5. Tipos de datos flotantes

Los tipos de datos flotantes permiten representar números con parte decimal. Los principales tipos disponibles en C++ son:

Tipo	Precisión aproximada
float	6 o 7 dígitos significativos
double	15 o 16 dígitos significativos
long double	Depende de la implementación

En programación competitiva normalmente se utiliza **double**, pues ofrece una precisión suficiente para la mayoría de los problemas.

```
double x = 3.141592653589793;
double y = 5.0 / 2.0;
```

Es importante que al menos uno de los operandos de una división sea flotante. De lo contrario, se realizará una división entera.

```
double x = 5 / 2;    // 2
double y = 5.0 / 2.0; // 2.5
```

La función **setprecision** permite controlar la cantidad de dígitos mostrados. Se encuentra en la biblioteca **iomanip**.

Sin utilizar **fixed**, indica el número total de dígitos significativos:

```
double pi = 3.141592653589793;
cout << setprecision(5) << pi;    // 3.1416
```

Al utilizar **fixed**, indica la cantidad de dígitos que se muestran después del punto decimal:

```
double pi = 3.141592653589793;
cout << fixed << setprecision(5) << pi;    // 3.14159
```

También puede utilizarse **scientific** para mostrar el número en notación científica:

```
double x = 123456.789;
cout << scientific << setprecision(3) << x;
// 1.235e+05
```

La biblioteca `math.h` contiene varias funciones matemáticas útiles.

Función	Resultado
<code>sqrt(x)</code>	Raíz cuadrada de x
<code>cbrt(x)</code>	Raíz cúbica de x
<code>pow(x,y)</code>	x^y
<code>abs(x)</code>	Valor absoluto de x
<code>floor(x)</code>	Mayor entero menor o igual que x
<code>ceil(x)</code>	Menor entero mayor o igual que x
<code>round(x)</code>	Entero más cercano a x
<code>trunc(x)</code>	Elimina la parte decimal
<code>exp(x)</code>	e^x
<code>log(x)</code>	Logaritmo natural de x
<code>log10(x)</code>	Logaritmo base 10 de x
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	Funciones trigonométricas

Por ejemplo:

```
double x = 9.0;
cout << sqrt(x) << "\n";    // 3
cout << pow(x, 2.0) << "\n"; // 81
cout << floor(3.8) << "\n";  // 3
cout << ceil(3.2) << "\n";   // 4
cout << round(3.6) << "\n";  // 4
```

Debido a errores de representación, no es recomendable comparar directamente dos números flotantes con el operador `==`.

```
double a = 0.1 + 0.2;
double b = 0.3;
const double EPS = 1e-9;
if (abs(a - b) < EPS) {
    cout << "Son aproximadamente iguales\n";
}
```

El valor de `EPS` depende de la precisión requerida por el problema, aunque valores como 10^{-9} o 10^{-12} son comunes al utilizar `double`.

2.6. Operaciones con bits

Los números enteros se almacenan en binario. C++ permite trabajar directamente con sus bits mediante los siguientes operadores:

Operador	Operación
<code>&</code>	AND bit a bit
<code> </code>	OR bit a bit
<code>^</code>	XOR bit a bit
<code>~</code>	Negación bit a bit
<code><<</code>	Desplazamiento a la izquierda
<code>>></code>	Desplazamiento a la derecha

Por ejemplo:

```
int a = 6;    // 110
```



```
int b = 3;    // 011
cout << (a & b) << "\n";    // 2: 010
cout << (a | b) << "\n";    // 7: 111
cout << (a ^ b) << "\n";    // 5: 101
```

Desplazar un número k posiciones a la izquierda equivale a multiplicarlo por 2^k , mientras que desplazarlo a la derecha equivale a dividirlo entre 2^k , ignorando el residuo:

```
cout << (5 << 2) << "\n";    // 20
cout << (20 >> 2) << "\n";    // 5
```

Para trabajar con el bit de posición i , se utiliza la máscara `1LL << i`:

```
long long x = 10;
int i = 1;

// Consultar el bit i
if (x & (1LL << i)) {
    cout << "El bit esta encendido\n";
}

// Encender el bit i
x |= (1LL << i);

// Apagar el bit i
x &= ~(1LL << i);

// Cambiar el bit i
x ^= (1LL << i);
```

La cantidad de bits encendidos puede obtenerse mediante:

```
int x = 13;

cout << __builtin_popcount(x) << "\n";    // 3
```

Para valores de tipo `long long`, se utiliza `__builtin_popcountll`.

3. Algoritmia y eficiencia

3.1. Límite en tiempo

Un procesador puede hacer aproximadamente 10^8 instrucciones por segundo, por eso es importante saber la complejidad de los algoritmos que vamos a utilizar, el tiempo que tenemos disponible también depende de cada problema. Como muchas complejidades dependen del tamaño de la entrada, en la siguiente tabla se muestra un pequeño resumen de cuál puede ser la complejidad máxima recomendada con respecto al tamaño de la entrada:

Tamaño de entrada n	Complejidad máxima recomendada	Comentario
10^2	$O(n^4)$	Todo pasa sin problema
10^3	$O(n^3)$	Clásico de fuerza bruta
10^4	$O(n^2)$	Ya empieza a ser el límite
10^5	$O(n \log(n))$	Ordenar, colas de prioridad, etc.
$> 10^6$	$O(n)$	Recorridos simples

Una manera “estándar” de saber la complejidad temporal del algoritmo que estás implementando en tu programa es preguntarte ¿de qué tamaño es la entrada?, para después preguntarte ¿cuántas veces estoy recorriendo lo equivalente al tamaño de la entrada? Si tu entrada fue una lista de n elementos, entonces la cantidad de for anidados de tu algoritmo es equivalente a la complejidad en tiempo, se puede ver fácilmente de la siguiente manera:

```

for(int i = 0; i < n; ++i) {    //Complejidad lineal O(n).

    for(int i = 0; i < n; ++i) {    //Complejidad cuadratica O(n^2).

        for(int i = 0; i < n; ++i) {    //Complejidad cubica O(n^3).
            ...
        }
    }
}

```

Existen complejidades un poco más complicadas, como la exponencial o la logarítmica, pero para usos prácticos de un curso de introducción a la programación competitiva, las polinomiales deberían de ser suficientes.

3.2. Límite en memoria

También, es importante el límite de memoria, puesto que declarar muchas variables puede ocupar más memoria de la que tenemos disponible. Debido a esto, se muestra una tabla con el tamaño de un arreglo de 10^6 elementos de los tipos de datos principales en C++.

Tipo	Tamaño por elemento	Memoria de un arreglo de 10^6 elementos
bool	1 byte	≈ 1 MB
short	2 bytes	≈ 2 MB
unsigned short	2 bytes	≈ 2 MB
int	4 bytes	≈ 4 MB
unsigned int	4 bytes	≈ 4 MB
long long / int64_t	8 bytes	≈ 8 MB
unsigned long long / uint64_t	8 bytes	≈ 8 MB
float	4 bytes	≈ 4 MB
double	8 bytes	≈ 8 MB
long double	16 bytes*	≈ 16 MB

Para un arreglo de mayor tamaño que 4 MB es necesario declararlo de manera global, es decir, fuera del main. La memoria disponible se puede consultar en cada problema.

4. Problemas clásicos de algoritmia sobre arreglos

4.1. Ocurrencias en una lista

Te dan una lista de N elementos, y después te dan M preguntas, de saber cuántas veces aparece el entero k en la lista de N elementos.

Entrada

Un entero N , seguido de N enteros que es la lista de elementos. Después un entero M seguido de M enteros k , que son las preguntas.

Salida

Para cada entero k , un entero que sea la cantidad de veces que aparece k en la lista de N elementos.

Ejemplo:

Entrada

```

5
1 3 1 5 3
4
1 2 3 5

```

Salida

```

2
0
2
1

```

Límites

$$0 \leq N, M \leq 10^6$$

$$1 \leq k \leq 10^7$$

Solución

Una forma poco eficiente de resolver este problema sería, para cada pregunta, recorrer toda la lista y contar cuántas veces aparece el número k . Si hay M preguntas, eso haría que repitiéramos el mismo trabajo muchas veces, dando una complejidad de $O(NM)$, lo cual puede ser demasiado lento.

La idea eficiente es usar un arreglo de ocurrencias. Como los valores de los elementos están en el rango de 1 a 10^7 , podemos crear un arreglo `ocurr` de tamaño $10^7 + 1$, donde en la posición `ocurr[x]` guardemos cuántas veces aparece el número x en la lista. Un arreglo de este tamaño, usando enteros de 4 bytes, ocupa aproximadamente 40 MB, así que todavía es manejable en memoria en muchos casos.

Entonces, en lugar de guardar la lista completa para contestar las preguntas después, mientras leemos los N elementos vamos aumentando su contador correspondiente. Por ejemplo, si leemos un 3, hacemos `ocurr[3]++`. Eso significa que el número 3 ha aparecido una vez más en la lista. Después, para cada pregunta con valor k , la respuesta ya está calculada: simplemente imprimimos `ocurr[k]`.

De esta manera, construir el arreglo de ocurrencias cuesta $O(N)$, responder las M preguntas cuesta $O(M)$, y la complejidad total es $O(N + M)$.

Código

El código se muestra a continuación*:

```
#include <iostream>
using namespace std;

int occur[10000000 + 1] = {};

int main() {
    int n;
    cin >> n;
    for(int i = 0; i < n; ++i) {
        int p;
        cin >> p;
        occur[p] += 1;
    }
    int m;
    cin >> m;
    for(int i = 0; i < m; ++i) {
        int k;
        cin >> k;
        cout << occur[k] << "\n";
    }
}
```

*Nota: Si N y M son muy grandes es necesario desincronizar `iostream` y `stdio`, aquí no se hizo para no hacer tan largo el programa. Además, si el rango de k es muy grande, el arreglo debe estar declarado fuera del `main`.

4.2. Sumas de prefijos en una lista

Te dan una lista de N elementos, y después te dan M preguntas, cada una formada por dos enteros l y r , que representan un intervalo de la lista. Para cada pregunta, debes saber cuál es la suma de los elementos del intervalo desde la posición l hasta la posición r .

Entrada

Un entero N , seguido de N enteros que forman la lista. Después un entero M , seguido de M pares de enteros l y r , que son los intervalos de las preguntas.

Salida

Para cada pregunta, imprimir un entero que sea la suma del intervalo l y r .

Ejemplo

Entrada:

```
5
2 4 1 3 5
4
1 3
2 5
3 3
4 5
```

Salida:

```
7
13
1
8
```

Límites

$1 \leq N, M \leq 10^6$

Solución

Una forma poco eficiente de resolver este problema sería, para cada pregunta, recorrer el intervalo desde la posición l hasta la posición r y sumar sus elementos. Si hacemos eso para las M preguntas, en el peor caso la complejidad puede llegar a ser $O(NM)$, lo cual puede ser demasiado lento.

La idea eficiente es usar un arreglo de sumas de prefijos. Construimos un arreglo `pref` donde `pref[i]` guarda la suma de los primeros i elementos de la lista. Es decir, `pref[0] = 0`, `pref[1]` guarda el primer elemento, `pref[2]` la suma de los dos primeros, y así sucesivamente. Con este arreglo, la suma de los elementos entre las posiciones l y r se puede obtener restando:

$$\text{pref}[r] - \text{pref}[l - 1]$$

Esto funciona porque `pref[r]` contiene la suma desde la posición 1 hasta la r , y al restarle `pref[l - 1]` eliminamos lo que había antes del intervalo que nos interesa.

De esta manera, construir el arreglo de prefijos cuesta $O(N)$, responder las M preguntas cuesta $O(M)$, y la complejidad total es $O(N + M)$.

Código

El código se muestra a continuación*:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long pref[1000000 + 1];
    pref[0] = 0;
    for(int i = 1; i <= n; ++i) {
        int x;
        cin >> x;
        pref[i] = pref[i - 1] + x;
    }
    int m;
    cin >> m;
    for(int i = 0; i < m; ++i) {
        int l, r;
        cin >> l >> r;
        cout << pref[r] - pref[l - 1] << "\n";
    }
}
```

*Nota: Aquí se usa `long long` en el arreglo de prefijos porque la suma de muchos elementos puede ser grande, aunque cada elemento individual no lo sea.

4.3. Problema de la mayor suma de un subarreglo de tamaño k

Te dan una lista de N enteros y un entero k . Debes encontrar la mayor suma posible de un subarreglo de tamaño exactamente k .

Entrada

Un entero N , seguido de N enteros que forman la lista. Después, un entero k .

Salida

Imprimir un entero que indique la mayor suma de un subarreglo continuo de tamaño k .

Ejemplo

Entrada

```
5
1 2 1 3 2
3
```

Salida

```
6
```

Límites

$$1 \leq k \leq N \leq 10^6$$

Solución

Una forma poco eficiente de resolver este problema sería revisar todos los subarreglos continuos de tamaño k y calcular su suma desde cero. Como hay $N - k + 1$ subarreglos de ese tamaño, y cada uno tarda $O(k)$ en sumarse, la complejidad total sería $O(N \cdot k)$, lo cual puede ser demasiado lento.

La idea eficiente es usar una ventana deslizante de tamaño fijo. Primero calculamos la suma de los primeros k elementos. Esa será la suma de la primera ventana. Después, para mover la ventana una posición a la derecha, ya no hace falta volver a sumar todo. Basta con restar el elemento que sale por la izquierda y sumar el nuevo elemento que entra por la derecha. Así obtenemos la suma de la nueva ventana en tiempo $O(1)$.

Repitiendo este proceso para todas las ventanas, podemos ir guardando la mayor suma encontrada. De esta manera, solo recorreremos la lista una vez, y la complejidad total es $O(N)$.

Código

El código se muestra a continuación:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int arr[1000000 + 1];
    for(int i = 0; i < n; ++i) {
        cin >> arr[i];
    }
    int k;
    cin >> k;
    long long suma = 0;
    for(int i = 0; i < k; ++i) {
        suma += arr[i];
    }
    long long mejor = suma;
    for(int i = k; i < n; ++i) {
        suma = suma + arr[i] - arr[i - k];
        if(suma > mejor) {
            mejor = suma;
        }
    }
    cout << mejor << "\n";
}
```

5. Apuntadores, la biblioteca `algorithm` y estructuras

5.1. Apuntadores

Un apuntador es una variable que guarda la dirección de memoria de otra variable. La sintaxis para declarar un apuntador es `tipo* nombre`, donde `tipo` es el tipo de dato al que apunta y `nombre` es el nombre del apuntador.

```
string* p;    //apuntador a cadena
```

Para guardar la dirección de memoria de una variable en un apuntador, se usa el operador `&`:

```
int x = 5;
int* p = &x;    //p guarda la direccion de memoria de x
```

Para obtener el valor al que apunta un apuntador, usamos el operador `*`:

```
cout << *p << "\n";    //imprime 5
*p = 10;                //ahora x vale 10
cout << x << "\n";    //imprime 10
```

Si un apuntador apunta a un elemento de un arreglo, podemos movernos en el arreglo sumando o restando posiciones:

```
int arr[5] = {2, 4, 5, 7, 9};
int* p = &arr[0];    //apunta al primer elemento del arreglo
cout << *p << "\n";    //imprime 2
p++;                //movemos el apuntador una posicion del arreglo
cout << *p << "\n";    //imprime 4
p += 3;            //movemos el apuntador tres posiciones del arreglo
cout << *p << "\n";    //imprime 9
```

También podemos restar apuntadores si ambos apuntan a elementos del mismo arreglo, el resultado es cuántas posiciones hay entre ellos:

```
int arr[5] = {1, 3, 5, 7, 8};
int* a = &arr[1];
int* b = &arr[4];
cout << b - a << "\n";    //imprime 3
```

La resta de apuntadores se usa mucho para obtener índices:

```
int arr[5] = {4, 8, 1, 9, 3};
int* p = max_element(&arr[0], &arr[0] + 5);
int pos = p - &arr[0];    //guarda el indice de la posicion a la que apunta p en el arreglo
cout << pos << "\n";    //imprime 3
```

5.2. Algunas funciones básicas de la biblioteca `algorithm`

A continuación, se muestra una tabla con las funciones básicas que tiene la biblioteca `algorithm` de C++.

Función	¿Para qué sirve?	Cómo se usa	Qué regresa
<code>max</code>	Mayor de dos valores	<code>max(a, b)</code>	el valor mayor
<code>min</code>	Menor de dos valores	<code>min(a, b)</code>	el valor menor
<code>swap</code>	Intercambia dos valores	<code>swap(a, b)</code>	no regresa nada
<code>reverse</code>	Invierte un arreglo o intervalo	<code>reverse(&arr[0], &arr[0] + n)</code>	no regresa nada
<code>max_element</code>	Mayor elemento de un arreglo	<code>max_element(&arr[0], &arr[0] + n)</code>	un apuntador al mayor
<code>min_element</code>	Menor elemento de un arreglo	<code>min_element(&arr[0], &arr[0] + n)</code>	un apuntador al menor
<code>count</code>	Cuenta ocurrencias	<code>count(&arr[0], &arr[0] + n, x)</code>	cuántas veces aparece
<code>find</code>	Busca un valor	<code>find(&arr[0], &arr[0] + n, x)</code>	apuntador a la posición

5.3. Paso por copia y por referencia

Al enviar una variable como argumento de una función, esta puede pasarse por **copia** o por **referencia**.

Esta elección es especialmente importante en programación competitiva. Pasar por copia un contenedor grande, como un **vector**, una **string** o un **map**, crea una copia completa cada vez que se llama a la función. Esto puede aumentar considerablemente el tiempo de ejecución y provocar *Time Limit Exceeded*.

5.3.1. Paso por copia

En el paso por copia, la función recibe una nueva variable con el mismo valor que el argumento original. Los cambios realizados dentro de la función no afectan a la variable original.

```
void duplicar(int x) {  
    x *= 2;  
}  
int n = 5;  
duplicar(n);  
cout << n << "\n";    // 5
```

El paso por copia es adecuado para tipos pequeños, como **int**, **char**, **bool** y **double**, pues copiarlos tiene un costo muy pequeño.

5.3.2. Paso por referencia

En el paso por referencia, el parámetro se declara utilizando el símbolo **&**. La función trabaja directamente con la variable original, por lo que puede modificarla.

```
void duplicar(int& x) {  
    x *= 2;  
}  
int n = 5;  
duplicar(n);  
cout << n << "\n";    // 10
```

Para modificar un contenedor dentro de una función, también debe pasarse por referencia:

```
void agregar(vector<int>& numeros, int x) {  
    numeros.push_back(x);  
}
```

5.3.3. Referencia constante

Cuando se quiere evitar una copia, pero no se desea modificar el argumento, se utiliza una referencia constante:

```
void imprimir(const vector<int>& numeros) {  
    for (int x : numeros) {  
        cout << x << " ";  
    }  
}
```

El símbolo **&** evita copiar todo el vector y la palabra **const** impide modificarlo dentro de la función. Por ejemplo, la siguiente función crea una copia completa del vector cada vez que se llama:

```
int suma(vector<int> numeros) {  
    int resultado = 0;  
    for (int x : numeros) {  
        resultado += x;  
    }  
    return resultado;  
}
```

Una forma más eficiente es:

```
int suma(const vector<int>& numeros) {
    int resultado = 0;
    for (int x : numeros) {
        resultado += x;
    }
    return resultado;
}
```

En programación competitiva, para contenedores y objetos grandes se recomienda:

- usar `tipo&` cuando la función debe modificar el objeto;
- usar `const tipo&` cuando solo debe consultarlo;
- usar `tipo` para tipos pequeños o cuando realmente se necesita una copia.

Forma	Crea una copia	Puede modificar el original
<code>tipo x</code>	Sí	No
<code>tipo& x</code>	No	Sí
<code>const tipo& x</code>	No	No

5.4. Recursión

Una función es **recursiva** cuando se llama a sí misma durante su ejecución. La recursión permite resolver un problema utilizando soluciones del mismo problema con entradas más pequeñas.

Toda función recursiva debe tener un **caso base**, es decir, una condición en la que la función termina sin realizar otra llamada recursiva. Por ejemplo, podemos calcular el factorial de un número de la siguiente manera:

```
long long factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Por ejemplo, al calcular `factorial(4)` se realizan las llamadas

$$4! = 4 \cdot 3! = 4 \cdot 3 \cdot 2! = 4 \cdot 3 \cdot 2 \cdot 1.$$

Cuando se alcanza el caso base, las llamadas comienzan a regresar sus resultados hasta obtener la respuesta final.

En general, una función recursiva tiene una estructura similar a

```
tipo funcion(parametros) {
    if (caso_base) {
        return resultado;
    }
    return funcion(entrada_mas_pequena);
}
```

Cada llamada recursiva se almacena temporalmente en la pila de ejecución. Por esta razón, una recursión demasiado profunda puede provocar un desbordamiento de pila (*stack overflow*).

La recursión es especialmente útil para trabajar con estructuras que pueden explorarse de manera recursiva, como árboles y gráficas. Por ejemplo, el recorrido en profundidad (DFS) utiliza una llamada recursiva para continuar el recorrido desde cada vértice encontrado.

5.5. Pares (pair)

Un `pair` permite almacenar dos valores, que pueden ser de tipos diferentes, dentro de una sola variable. Su declaración general es:

```
pair<T1, T2> nombre;
```

Por ejemplo:

```
pair<int, string> persona = {20, "Ana"};
cout << persona.first << "\n";    // 20
cout << persona.second << "\n";   // Ana
```

Los elementos se consultan mediante `first` y `second`. También pueden modificarse directamente:

```
persona.first = 21;
persona.second = "Luis";
```

Otra forma de crear un par es mediante `make_pair`:

```
pair<int, int> punto = make_pair(3, 5);
```

También es posible separar sus elementos utilizando `auto`:

```
auto [x, y] = punto;
cout << x << " " << y << "\n";    // 3 5
```

Los pares se comparan lexicográficamente: primero se compara `first` y, en caso de empate, se compara `second`. Por esta razón, un vector de pares puede ordenarse directamente:

```
vector<pair<int, int>> puntos = {
    {2, 5}, {1, 7}, {2, 3}
};
sort(puntos.begin(), puntos.end());
```

Después de ordenar, los pares quedan como (1, 7), (2, 3), (2, 5). Los pares son especialmente útiles para representar coordenadas, aristas o valores acompañados de su posición original.

5.6. Declaración de estructuras

En C++, una estructura es muy parecida a una clase, pero sus atributos y funciones son públicos de manera predeterminada, un ejemplo de una estructura que guarda los datos de un alumno puede ser la siguiente:

```
struct alumno {
    string nombre;
    int edad;
    int matricula;
};
```

Podemos declarar un “objeto” de la estructura de la siguiente manera:

```
alumno ejemplo = {"Pedro", 12, 22222222};    //Se debe de declarar en el mismo orden
```

Para acceder a un atributo de la estructura, se hace de la siguiente manera:

```
cout << ejemplo.nombre << "\n";    //Imprime "Pedro"
```

Si queremos acceder a un atributo de una estructura, de la cual tenemos guardada su dirección de memoria en un apuntador, se puede hacer de dos formas: desapuntando el apuntador entre paréntesis, o usando el operador “flechita”:

```
alumno* p = &ejemplo;
cout << (*p).nombre << "\n";    //Imprime Pedro
cout << p->nombre << "\n";    //Imprime Pedro tambien
```

6. Algoritmos de ordenamiento y de búsqueda de secuencias

6.1. Ordenamiento de secuencias

Para ordenar elementos en C++, normalmente usamos la función `sort`, que se encuentra en la biblioteca `algorithm`. Esta función recibe dos apuntadores o iteradores, que indican el inicio y el final del intervalo que queremos ordenar. Nosotros vamos a considerar el final del intervalo como la posición siguiente al último elemento del intervalo. Por ejemplo, si queremos ordenar un arreglo `arr` de tamaño n , escribimos:

```
sort(&arr[0], &arr[0] + n);
```

Aquí `&arr[0]` apunta al primer elemento del arreglo, y `&arr[0] + n` apunta a la posición siguiente al último elemento, es decir al final del arreglo. De manera predeterminada, `sort` ordena de menor a mayor:

```
int arr[5] = {4, 1, 3, 5, 2};
sort(&arr[0], &arr[0] + 5);
```

Después de esto, el arreglo queda así:

```
1 2 3 4 5
```

La complejidad de `sort` es, en general, $O(n \log(n))$, por lo que es muy eficiente para arreglos grandes. También es posible indicar una forma distinta de ordenar usando un predicado. Un predicado es una función que recibe dos elementos y decide cuál debe de ir primero. El predicado debe regresar `true` si el primer elemento debe de ir antes que el segundo, y `false` en caso contrario. Por ejemplo, si queremos ordenar una lista de enteros de mayor a menor, podemos hacer lo siguiente:

```
bool mayor(int& a, int& b) {
    return a > b;
}
```

Esta función, en general, se debe de declarar fuera del `main`, y nosotros la mandamos a llamar en la misma función de `sort`, de la siguiente manera:

```
sort(&arr[0], &arr[0] + n, mayor);
```

También podemos decidir cómo ordenar listas de estructuras u objetos usando predicados. Por ejemplo, para la estructura `alumno` de la sección anterior, si queremos ordenar un arreglo de alumnos, primero por edad, y en caso de empate, alfabéticamente por nombre, podemos declarar el siguiente predicado:

```
bool comp(alumno& a, alumno& b) {
    if(a.edad != b.edad) {
        return a.edad < b.edad;
    }
    return a.nombre < b.nombre;
}
```

Aquí, primero revisamos si las edades son distintas. Si lo son, ponemos primero al alumno con menor edad. Si las edades son iguales, entonces comparamos los nombres en orden alfabético.

6.2. Búsqueda eficiente sobre secuencias ordenadas

La búsqueda binaria permite buscar elementos eficientemente en una secuencia ordenada. Para una secuencia de tamaño n , su complejidad es $O(\log n)$. La biblioteca `algorithm` proporciona las funciones `binary_search`, `lower_bound` y `upper_bound`. Todas ellas reciben el inicio y el final del intervalo en el que se realizará la búsqueda. El inicio está incluido, pero el final no.

`binary_search`. Indica si un elemento se encuentra en la secuencia y regresa un valor de tipo `bool`.

```
int arr[5] = {1, 2, 5, 7, 8};
int x = 7;
bool existe = binary_search(arr, arr + 5, x);
cout << existe << "\n";    // 1
```

La función regresa **true** si el elemento existe y **false** en otro caso.

lower_bound. Regresa un apuntador a la primera posición cuyo valor es mayor o igual que el valor buscado. **upper_bound**. Regresa un apuntador a la primera posición cuyo valor es estrictamente mayor que el valor buscado.

Función	¿Para qué sirve?	Cómo se usa	Qué regresa
<code>binary_search</code>	Verifica si existe x	<code>binary_search(arr, arr + n, x)</code>	true o false
<code>lower_bound</code>	Busca la primera posición con valor $\geq x$	<code>lower_bound(arr, arr + n, x)</code>	Apuntador a la primera posición $\geq x$
<code>upper_bound</code>	Busca la primera posición con valor $> x$	<code>upper_bound(arr, arr + n, x)</code>	Apuntador a la primera posición $> x$

Por ejemplo:

```
int arr[5] = {1, 2, 5, 7, 8};
int* izq = lower_bound(arr, arr + 5, 7);
int* der = upper_bound(arr, arr + 5, 7);
cout << *izq << " " << *der << "\n";    // 7 8
```

Si no existe una posición que cumpla la condición, ambas funciones pueden regresar el final del intervalo. Por ello, antes de acceder al valor conviene verificarlo:

```
int* pos = lower_bound(arr, arr + 5, 10);
if (pos == arr + 5) {
    cout << "No existe una posicion adecuada\n";
}
```

También pueden utilizarse para contar cuántas veces aparece un elemento:

```
int arr[7] = {1, 2, 2, 2, 5, 7, 8};
int cantidad = upper_bound(arr, arr + 7, 2) - lower_bound(arr, arr + 7, 2);
cout << cantidad << "\n";    // 3
```

Estas funciones también pueden emplearse con estructuras y otros tipos de datos, siempre que se proporcione el mismo comparador utilizado para ordenar la secuencia:

```
alumno arr[n];
// El arreglo debe estar ordenado usando comp.
alumno x = {"Pedro", 12, 22222222};
bool existe = binary_search(arr, arr + n, x, comp);
cout << existe << "\n";
```

7. Estructuras de datos lineales

7.1. Cadenas de caracteres (string)

Una **string** es un tipo de dato de la biblioteca estándar de C++ que permite almacenar y manipular texto. Una cadena se declara de la siguiente manera:

```
string cad;
string nombre = "Pedro";
```

Las cadenas utilizan comillas dobles, mientras que los caracteres individuales utilizan comillas simples:

```
string cad = "hola";
char letra = 'h';
```

Para leer una palabra se utiliza **cin**. La lectura se detiene al encontrar un espacio:

```
string cad;
cin >> cad;
```

Para leer una línea completa, incluyendo espacios, se utiliza **getline**:

```
string cad;
getline(cin, cad);
```

Si se utiliza `getline` después de una lectura con `cin`, es necesario eliminar primero el salto de línea pendiente:

```
int n;
string cad;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
getline(cin, cad);
```

Los caracteres se numeran desde 0 y pueden consultarse o modificarse utilizando corchetes:

```
string cad = "hola";
cout << cad[0] << "\n";    // Imprime h
cad[0] = 'H';
cout << cad << "\n";      // Imprime Hola
```

También es posible concatenar cadenas y agregar caracteres al final:

```
string cad = "ho";
cad.push_back('l');        // "hol"
cad += "a";                // "hola"
cad = cad + " mundo";      // "hola mundo"
```

Las cadenas pueden compararse con los operadores `==`, `!=`, `<`, `>` y los demás operadores de comparación. El orden utilizado es lexicográfico:

```
string a = "ana";
string b = "luis";
if (a < b) {
    cout << "ana va antes que luis\n";
}
```

A continuación, se presentan las funciones más utilizadas:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>size</code>	Obtiene la longitud	<code>cad.size()</code>	Cantidad de caracteres
<code>empty</code>	Verifica si está vacía	<code>cad.empty()</code>	true o false
<code>clear</code>	Borra todo el contenido	<code>cad.clear()</code>	No regresa nada
<code>front</code>	Accede al primer carácter	<code>cad.front()</code>	El primer carácter
<code>back</code>	Accede al último carácter	<code>cad.back()</code>	El último carácter
<code>push_back</code>	Agrega un carácter al final	<code>cad.push_back(c)</code>	No regresa nada
<code>pop_back</code>	Elimina el último carácter	<code>cad.pop_back()</code>	No regresa nada
<code>substr</code>	Obtiene una subcadena	<code>cad.substr(i, len)</code>	Subcadena desde <code>i</code> de longitud <code>len</code>
<code>find</code>	Busca un carácter o una cadena	<code>cad.find("abc")</code>	Posición encontrada o string::npos
<code>erase</code>	Elimina una parte de la cadena	<code>cad.erase(i, len)</code>	Elimina desde <code>i</code>
<code>stoi</code>	Convierte una cadena a int	<code>stoi(cad)</code>	Un valor de tipo int
<code>stoll</code>	Convierte una cadena a long long	<code>stoll(cad)</code>	Un valor de tipo long long
<code>stod</code>	Convierte una cadena a double	<code>stod(cad)</code>	Un valor de tipo double
<code>to_string</code>	Convierte un número a cadena	<code>to_string(x)</code>	Una string que representa a <code>x</code>

Por ejemplo, las conversiones entre cadenas y números pueden realizarse de la siguiente manera:

```
string cad = "12345";

int x = stoi(cad);
long long y = stoll(cad);
double z = stod("3.1416");
```

```
string a = to_string(x);
```

7.2. Arreglos dinámicos (vector)

Un **vector** es una estructura de la biblioteca estándar de C++ que funciona como un arreglo dinámico. A diferencia de un arreglo normal, su tamaño puede cambiar durante la ejecución del programa. La sintaxis general para declarar un vector es `vector<tipo>nombre`, por ejemplo:

```
vector<int> v;           //vector de enteros
vector<string> nombres; //vector de cadenas
```

También podemos declarar un vector con un tamaño inicial:

```
vector<int> v(5); //vector de 5 enteros
```

Si queremos declarar un vector, con un tamaño inicial y con todos sus elementos con cierto valor, lo hacemos de la siguiente forma:

```
vector<int> v(5, -1); //vector de 5 enteros inicializados con -1
```

Para acceder a un elemento, usamos corchetes igual que en los arreglos:

```
cout << v[0] << "\n";
```

A continuación, se presenta una tabla con las funciones básicas de **vector**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>push_back</code>	Agrega un elemento al final	<code>v.push_back(x)</code>	agrega x al final
<code>pop_back</code>	Quita el último elemento	<code>v.pop_back()</code>	no regresa nada
<code>size</code>	Dice cuántos elementos tiene el vector	<code>v.size()</code>	la cantidad de elementos
<code>empty</code>	Verifica si el vector está vacío	<code>v.empty()</code>	true o false
<code>back</code>	Da el último elemento	<code>v.back()</code>	el último elemento
<code>front</code>	Da el primer elemento	<code>v.front()</code>	el primer elemento

Si queremos un apuntador al inicio o al final del vector, podemos usar las funciones `begin()` y `end()`. Por ejemplo, si queremos ordenar un vector:

```
sort(v.begin(), v.end());
```

7.3. Pilas (stack)

Una **stack** es una estructura de la biblioteca estándar de C++ que sigue la idea de último en entrar, primero en salir. Es decir, el último elemento que se mete a la pila es el primero que se puede sacar. La sintaxis general para declarar una pila es `stack<tipo>nombre`. Por ejemplo:

```
stack<int> pila;
stack<string> nombres;
```

A continuación, se presenta una tabla con las funciones básicas de **stack**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>push</code>	Mete un elemento a la pila	<code>pila.push(x)</code>	agrega x arriba
<code>pop</code>	Quita el elemento de arriba	<code>pila.pop()</code>	no regresa nada
<code>top</code>	Da el elemento de arriba	<code>pila.top()</code>	el elemento superior
<code>size</code>	Dice cuántos elementos tiene la pila	<code>pila.size()</code>	la cantidad de elementos
<code>empty</code>	Verifica si la pila está vacía	<code>pila.empty()</code>	true o false

Una observación importante es que `pop()` **no regresa** el elemento que quita. Si quieres guardar ese valor, primero debes usar `top()` y después `pop()`.

```
int x = pila.top();
pila.pop();
```

7.4. Colas (queue)

Una **queue** es una estructura de la biblioteca estándar de C++ que sigue la idea de primero en entrar, primero en salir. Es decir, el primer elemento que entra a la cola es el primero que sale. La sintaxis general para declarar una cola es `queue<tipo>nombre`. Por ejemplo:

```
queue<int> cola;
queue<string> nombres;
```

A continuación, se presenta una tabla con las funciones básicas de **queue**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push	Mete un elemento a la cola	cola.push(x)	agrega x al final
pop	Quita el elemento del frente	cola.pop()	no regresa nada
front	Da el elemento del frente	cola.front()	el primer elemento
back	Da el elemento del final	cola.back()	el último elemento
size	Dice cuántos elementos tiene la cola	cola.size()	la cantidad de elementos
empty	Verifica si la cola está vacía	cola.empty()	true o false

Al igual que en **stack**, `pop()` **no regresa** el elemento que quita. Si quieres guardarlo, primero debes usar `front()` y después `pop()`.

```
int x = cola.front();
cola.pop();
```

7.5. Dobles colas (deque)

Un **deque** es una estructura de la biblioteca estándar de C++ parecida a un vector, pero con la ventaja de que permite agregar y quitar elementos tanto al inicio como al final de manera eficiente. La palabra **deque** viene de *double ended queue*, es decir, una cola con dos extremos. La sintaxis general para declarar un **deque** es `deque<tipo>nombre`. Por ejemplo:

```
deque<int> d;
deque<string> nombres;
```

También podemos acceder a cualquier posición usando corchetes, igual que en un arreglo o en un vector:

```
cout << d[1] << "\n";
```

A continuación, se presenta una tabla con las funciones básicas de **deque**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push_back	Agrega un elemento al final	d.push_back(x)	agrega x al final
push_front	Agrega un elemento al inicio	d.push_front(x)	agrega x al inicio
pop_back	Quita el último elemento	d.pop_back()	no regresa nada
pop_front	Quita el primer elemento	d.pop_front()	no regresa nada
front	Da el primer elemento	d.front()	el primer elemento
back	Da el último elemento	d.back()	el último elemento
size	Dice cuántos elementos tiene	d.size()	la cantidad de elementos
empty	Verifica si está vacío	d.empty()	true o false

7.6. Listas enlazadas (list)

Una `list` es una estructura de la biblioteca estándar de C++ que permite agregar y quitar elementos de manera eficiente, tanto al inicio como al final, y también insertar o borrar elementos en posiciones intermedias si ya tenemos un iterador a esa posición. La sintaxis general para declarar una lista es `list<tipo>nombre`. Por ejemplo:

```
list<int> lista;
list<string> nombres;
```

Como una `list` no se recorre con índices, normalmente se usa un iterador:

```
for(list<int>::iterator it = lista.begin(); it != lista.end(); ++it) {
    cout << *it << " ";
}
```

Si tenemos un iterador a una posición de la lista, podemos agregar o eliminar elementos en esa posición de la siguiente manera:

```
list<int> lista;
lista.push_back(10);
lista.push_back(20);
list<int>::iterator it = lista.begin(); ++it;
lista.insert(it, 15);
lista.erase(it);
```

A continuación, se presenta una tabla con las funciones básicas de `list`:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>push_back</code>	Agrega un elemento al final	<code>lista.push_back(x)</code>	agrega x al final
<code>push_front</code>	Agrega un elemento al inicio	<code>lista.push_front(x)</code>	agrega x al inicio
<code>pop_back</code>	Quita el último elemento	<code>lista.pop_back()</code>	no regresa nada
<code>pop_front</code>	Quita el primer elemento	<code>lista.pop_front()</code>	no regresa nada
<code>front</code>	Da el primer elemento	<code>lista.front()</code>	el primer elemento
<code>back</code>	Da el último elemento	<code>lista.back()</code>	el último elemento
<code>insert</code>	Inserta un elemento antes de una posición	<code>lista.insert(it, x)</code>	inserta x antes de it
<code>erase</code>	Borra el elemento en una posición	<code>lista.erase(it)</code>	borra el elemento apuntado por it
<code>size</code>	Dice cuántos elementos tiene	<code>lista.size()</code>	la cantidad de elementos
<code>empty</code>	Verifica si está vacía	<code>lista.empty()</code>	true o false
<code>begin</code>	Da un iterador al inicio	<code>lista.begin()</code>	iterador al primer elemento
<code>end</code>	Da un iterador al final lógico	<code>lista.end()</code>	iterador a la posición siguiente al último elemento

7.7. Ciclo for-each

El ciclo `for-each` permite recorrer directamente todos los elementos de un arreglo o contenedor, sin utilizar índices. Su sintaxis general es:

```
for (tipo elemento : contenedor) {
    // Instrucciones
}
```

Por ejemplo, para recorrer un `vector`:

```
vector<int> numeros = {4, 7, 2, 9};

for (int x : numeros) {
    cout << x << " ";    //4 7 2 9
}
```

En cada iteración, `x` recibe una copia del siguiente elemento. Por esta razón, modificar `x` no modifica el contenedor original:

```
for (int x : numeros) {
    x *= 2;
```

```
}
```

Para modificar directamente los elementos, se utiliza una referencia:

```
for (int& x : numeros) {  
    x *= 2;  
}
```

También se puede utilizar `auto` para que C++ determine automáticamente el tipo de dato:

```
for (auto x : numeros) {  
    cout << x << " ";  
}
```

Cuando solamente se desean consultar los elementos, especialmente si son objetos grandes, se puede utilizar una referencia constante:

```
vector<string> nombres = {"Ana", "Luis", "Pedro"};  
  
for (const auto& nombre : nombres) {  
    cout << nombre << "\n";  
}
```

El ciclo también puede utilizarse para recorrer cadenas de caracteres:

```
string cad = "hola";  
  
for (char c : cad) {  
    cout << c << "\n";  
}
```

8. Estructuras de datos no lineales

8.1. Conjuntos (set)

Un **set** es una estructura de la biblioteca estándar de C++ que guarda elementos **ordenados** y **sin repetir**. La sintaxis general para declarar un **set** es `set<tipo>nombre`. Por ejemplo:

```
set<int> s;  
set<string> nombres;
```

Para agregar elementos, usamos `insert`:

```
s.insert(10);  
s.insert(5);  
s.insert(20);  
s.insert(10);
```

Aunque se intente insertar 10 dos veces, en el **set** solo aparecerá una vez. Para recorrer un **set**, usamos iteradores:

```
for(set<int>::iterator it = s.begin(); it != s.end(); ++it) {  
    cout << *it << " ";  
}
```

Para eliminar elementos, si existen, usamos `erase`:

```
s.erase(5);
```

A continuación, se presenta una tabla con las funciones básicas de **set**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>insert</code>	Agrega un elemento	<code>s.insert(x)</code>	inserta <code>x</code> si no estaba
<code>erase</code>	Borra un elemento	<code>s.erase(x)</code>	borra <code>x</code> si existe
<code>count</code>	Verifica si un elemento existe	<code>s.count(x)</code>	1 o 0
<code>find</code>	Busca un elemento	<code>s.find(x)</code>	iterador al elemento o <code>s.end()</code>
<code>size</code>	Dice cuántos elementos tiene	<code>s.size()</code>	la cantidad de elementos
<code>empty</code>	Verifica si está vacío	<code>s.empty()</code>	<code>true</code> o <code>false</code>
<code>begin</code>	Da un iterador al inicio	<code>s.begin()</code>	iterador al menor elemento
<code>end</code>	Da un iterador al final lógico	<code>s.end()</code>	iterador a la posición siguiente al último

8.2. Diccionarios (map)

Un `map` es una estructura que guarda pares de la forma **clave-valor**. Las claves están ordenadas y no se repiten. La sintaxis general es `map<tipo_clave, tipo_valor>nombre`. Por ejemplo:

```
map<string, int> edad;
```

Aquí la clave es un `string` y el valor es un `int`. Podemos guardar valores así:

```
edad["Ana"] = 18;
edad["Luis"] = 20;
edad["Carlos"] = 19;
```

Podemos acceder al valor asociado a una clave de la siguiente manera:

```
string nombre = "Ana";
cout << edad[nombre] << "\n";    //Imprime 18
```

Podemos recorrer un `map` usando iteradores. Aquí, `(*it).first` es la clave y `(*it).second` es el valor:

```
for(map<string, int>::iterator it = edad.begin(); it != edad.end(); ++it) {
    cout << (*it).first << " " << (*it).second << "\n";
}
```

A continuación, se presenta una tabla con las funciones básicas de `map`:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>insert</code>	Inserta un par clave-valor	<code>m.insert({a, b})</code>	inserta el par {clave, valor} si la clave no estaba
<code>erase</code>	Borra una clave	<code>m.erase(x)</code>	borra la clave <code>x</code> si existe
<code>contains</code>	Verifica si una clave existe	<code>m.contains(x)</code>	1 o 0
<code>find</code>	Busca una clave	<code>m.find(x)</code>	iterador a la clave o <code>m.end()</code>
<code>size</code>	Dice cuántos pares tiene	<code>m.size()</code>	la cantidad de pares
<code>empty</code>	Verifica si está vacío	<code>m.empty()</code>	<code>true</code> o <code>false</code>
<code>begin</code>	Da un iterador al inicio	<code>m.begin()</code>	iterador al primer par
<code>end</code>	Da un iterador al final lógico	<code>m.end()</code>	iterador a la posición siguiente al último

8.3. Colas de prioridad (priority_queue)

Una `priority_queue` es una estructura parecida a una cola, pero en lugar de sacar el primero que entró, saca primero el elemento de **mayor prioridad**.

En C++, por defecto, una `priority_queue` saca primero el elemento **más grande**. Se encuentra en la biblioteca `queue`.

La sintaxis general es `priority_queue<tipo>nombre`. Por ejemplo:

```
priority_queue<int> pq;
```

A continuación, se muestra una lista de las funciones básicas de `priority_queue`:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push	Mete un elemento	pq.push(x)	agrega x a la estructura
pop	Quita el elemento de mayor prioridad	pq.pop()	no regresa nada
top	Da el elemento de mayor prioridad	pq.top()	el elemento superior
size	Dice cuántos elementos tiene	pq.size()	la cantidad de elementos
empty	Verifica si está vacía	pq.empty()	true o false

Si queremos usar una `priority_queue` con una estructura creada por nosotros, o simplemente queremos ordenar de otra forma, C++ necesita saber cómo comparar dos elementos para decidir cuál tiene mayor prioridad. Para eso, podemos **sobrecargar el operador <** dentro de la estructura, usando la misma idea que los predicados de ordenamiento:

```
struct alumno {
    string nombre;
    int edad;
    int matricula;
    bool operator<(const alumno& otro) const {
        if(edad != otro.edad) {
            return edad < otro.edad;
        }
        return nombre > otro.nombre;
    }
};
```

Aquí, el alumno con mayor edad sigue teniendo mayor prioridad. Si dos alumnos tienen la misma edad, quedará arriba el que tenga el nombre alfabéticamente menor. Ya teniendo sobrecargado el operador, nosotros inicializamos la cola de prioridad de la misma manera:

```
priority_queue<alumno> pq;
pq.push({"Pedro", 20, 2222222});
```

8.4. Gráficas

Una gráfica es una estructura formada por un conjunto de **vértices** y un conjunto de **aristas** que conectan pares de vértices. En programación, las representaciones más comunes de una gráfica son la lista de adyacencia y la matriz de adyacencia.

Supongamos que la gráfica tiene n vértices, numerados desde 0 hasta $n - 1$, y m aristas.

8.4.1. Lista de adyacencia

Una lista de adyacencia guarda, para cada vértice, una lista con todos sus vecinos. Se puede declarar como un arreglo de vectores:

```
vector<int> lista[n];
```

De esta manera, `lista[u]` contiene todos los vértices adyacentes al vértice `u`.

Si la entrada proporciona las aristas de una gráfica no dirigida, cada arista debe agregarse en ambos sentidos:

```
int n, m;
cin >> n >> m;
vector<int> lista[n];
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    lista[u].push_back(v);
    lista[v].push_back(u);
}
```

Por ejemplo, si se recibe la arista 2-5, entonces 5 se agrega a la lista de vecinos de 2 y 2 se agrega a la lista de vecinos de 5.

Para recorrer los vecinos de un vértice u :

```
for (int v : lista[u]) {
    cout << v << " ";
}
```

Para mostrar los vecinos de todos los vértices:

```
for (int u = 0; u < n; ++u) {
    cout << u << ": ";
    for (int v : lista[u]) {
        cout << v << " ";
    }
    cout << "\n";
}
```

En una gráfica dirigida, una arista $u \rightarrow v$ se agrega solamente a la lista del vértice de origen:

```
lista[u].push_back(v);
```

La lista de adyacencia utiliza memoria $O(n + m)$. Recorrer los vecinos de un vértice u tiene complejidad $O(\text{grado}(u))$.

8.4.2. Matriz de adyacencia

Una matriz de adyacencia es una matriz de tamaño $n \times n$, donde la posición `matriz[u][v]` indica si existe una arista entre los vértices u y v .

$$\text{matriz}[u][v] = \begin{cases} \text{true}, & \text{si existe la arista,} \\ \text{false}, & \text{en otro caso.} \end{cases}$$

Se puede declarar de la siguiente manera:

```
vector<vector<bool>> matriz(n, vector<bool>(n, false));
```

Si la entrada proporciona las aristas de una gráfica no dirigida:

```
int n, m;
cin >> n >> m;
vector<vector<bool>> matriz(n, vector<bool>(n, false));
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    matriz[u][v] = true;
    matriz[v][u] = true;
}
```

Para saber si existe una arista entre dos vértices:

```
if (matriz[u][v]) {
    cout << "Existe una arista\n";
}
```

Para recorrer los vecinos de un vértice u , se revisa toda la fila correspondiente:

```
for (int v = 0; v < n; ++v) {
    if (matriz[u][v]) {
        cout << v << " ";
    }
}
```

Para mostrar los vecinos de todos los vértices:

```
for (int u = 0; u < n; ++u) {
    cout << u << ": ";
    for (int v = 0; v < n; ++v) {
        if (matriz[u][v]) {
            cout << v << " ";
        }
    }

    cout << "\n";
}
```

En una gráfica dirigida, una arista $u \rightarrow v$ se guarda únicamente como:

```
matriz[u][v] = true;
```

La matriz de adyacencia utiliza memoria $O(n^2)$. Consultar si existe una arista entre dos vértices tiene complejidad $O(1)$, mientras que recorrer los vecinos de un vértice tiene complejidad $O(n)$.

8.4.3. Comparación de representaciones

Operación	Lista de adyacencia	Matriz de adyacencia
Memoria utilizada	$O(n + m)$	$O(n^2)$
Agregar una arista	$O(1)$	$O(1)$
Consultar si existe la arista u, v	$O(\text{grado}(u))$	$O(1)$
Recorrer los vecinos de u	$O(\text{grado}(u))$	$O(n)$

La lista de adyacencia suele ser la representación más utilizada en programación competitiva, especialmente cuando la gráfica tiene pocas aristas. La matriz de adyacencia es conveniente cuando la gráfica es pequeña y se necesita consultar frecuentemente si existe una arista entre dos vértices.

8.4.4. Recorrido en amplitud (BFS)

Una vez que tenemos almacenada una gráfica, una de las operaciones más comunes es recorrer sus vértices. El **recorrido en amplitud**, conocido como **BFS** por *Breadth-First Search*, comienza en un vértice y visita primero todos sus vecinos, después los vecinos de estos y así sucesivamente. De esta manera, la gráfica se recorre por niveles de distancia respecto al vértice inicial.

Para mantener el orden en que deben procesarse los vértices se utiliza una cola. Además, podemos guardar para cada vértice su distancia mínima al vértice inicial. Inicialmente asignamos distancia -1 a todos los vértices para indicar que todavía no han sido visitados.

```
vector<int> distancia(n, -1);
queue<int> cola;
distancia[inicio] = 0;
cola.push(inicio);
while (!cola.empty()) {
    int actual = cola.front();
    cola.pop();
    for (int vecino : lista[actual]) {
        if (distancia[vecino] == -1) {
            distancia[vecino] = distancia[actual] + 1;
            cola.push(vecino);
        }
    }
}
```

Cuando un vértice v es encontrado por primera vez, su distancia es una unidad mayor que la del vértice desde el cual fue alcanzado. BFS garantiza que esta es la distancia mínima en número de aristas desde **inicio** hasta v .

El arreglo **distancia** también nos permite determinar qué vértices son alcanzables desde el vértice inicial. Si al terminar el algoritmo algún vértice conserva el valor -1 , entonces no existe un camino desde **inicio** hasta él. En particular, si la gráfica es no dirigida, será conexa si y solo si ningún vértice queda con distancia -1 .

Utilizando una lista de adyacencia, cada vértice se procesa una vez y cada arista se examina una cantidad constante de veces, por lo que la complejidad de BFS es

$$O(n + m).$$

8.4.5. Recorrido en profundidad (DFS)

El **recorrido en profundidad**, conocido como **DFS** por *Depth-First Search*, sigue una estrategia diferente. Al llegar a un vértice, intenta continuar tanto como sea posible a través de alguno de sus vecinos antes de regresar y explorar otras posibilidades.

Una forma natural de implementar este comportamiento es mediante recursión. La función recibe el vértice que queremos visitar, lo marca y después realiza el mismo procedimiento sobre cada uno de sus vecinos que todavía no haya sido visitado.

```
vector<bool> visitado(n, false);
void dfs(int actual) {
    visitado[actual] = true;
    for (int vecino : lista[actual]) {
        if (!visitado[vecino]) {
            dfs(vecino);
        }
    }
}

dfs(inicio);
```

Para comenzar el recorrido desde un vértice determinado:

```
dfs(inicio);
```

Por ejemplo, si desde el vértice actual existen varios vecinos sin visitar, DFS seleccionará uno de ellos y continuará recorriendo la gráfica desde ahí. Cuando ya no pueda avanzar, regresará al vértice anterior y continuará con los vecinos pendientes.

El orden exacto en el que BFS y DFS visitan los vértices depende del orden en el que los vecinos estén almacenados en la lista de adyacencia. Por esta razón, pueden existir diferentes recorridos BFS o DFS válidos para una misma gráfica.

Al igual que BFS, cuando se utiliza una lista de adyacencia, DFS tiene complejidad

$$O(n + m),$$

mientras que con una matriz de adyacencia su complejidad es $O(n^2)$.

DFS es una herramienta fundamental para muchos algoritmos sobre gráficas. Entre otras cosas, se utiliza como base para encontrar componentes conexas, detectar ciclos, realizar ordenamientos topológicos y estudiar diferentes propiedades de conectividad.

8.4.6. Gráficas no conexas

Tanto BFS como DFS, cuando comienzan desde un único vértice, recorren solamente los vértices que son alcanzables desde él. Por lo tanto, si la gráfica no es conexa, algunos vértices quedarán sin visitar.

En el caso de DFS, para recorrer toda la gráfica podemos iniciar un nuevo recorrido cada vez que encontremos un vértice que todavía no haya sido visitado:

```
for (int inicio = 0; inicio < n; ++inicio) {
    if (!visitado[inicio]) {
        dfs(inicio);
    }
}
```

Cada llamada a `dfs(inicio)` realizada desde el ciclo principal comienza el recorrido de una nueva componente conexa. Por lo tanto, podemos contar las componentes de la gráfica añadiendo un contador:

```
vector<bool> visitado(n, false);
int componentes = 0;
for (int inicio = 0; inicio < n; ++inicio) {
    if (!visitado[inicio]) {
        ++componentes;
        dfs(inicio);
    }
}
```

En particular, una gráfica no dirigida es conexa si y solo si `componentes == 1`.

8.4.7. Comparación entre BFS y DFS

Aunque ambos algoritmos permiten recorrer todos los vértices alcanzables desde un vértice inicial y tienen la misma complejidad utilizando listas de adyacencia, el orden en que realizan el recorrido es diferente.

	BFS	DFS
Estrategia	Recorre por niveles	Avanza en profundidad
Estructura utilizada	Cola	Recursión o pila
Complejidad con lista de adyacencia	$O(n + m)$	$O(n + m)$
Distancias mínimas sin pesos	Sí	No necesariamente

En general, BFS es especialmente útil cuando importa la distancia mínima en número de aristas desde un vértice inicial, mientras que DFS aparece frecuentemente en problemas relacionados con conectividad y con la estructura de la gráfica.

9. Algunos algoritmos aritméticos básicos

9.1. Operaciones con módulo

En programación competitiva es común que el resultado de un problema sea muy grande. Para evitar desbordamientos, a veces el enunciado pide imprimir la respuesta módulo algún número, por ejemplo $10^9 + 7$. Si tenemos dos números a y b , y un módulo m , las operaciones básicas se pueden hacer así:

$$(a + b) \% m = ((a \% m) + (b \% m)) \% m$$

$$(a - b) \% m = ((a \% m) - (b \% m) + m) \% m$$

$$(a \cdot b) \% m = ((a \% m) \cdot (b \% m)) \% m$$

La idea es que podemos ir tomando módulo en cada paso para que los números no crezcan demasiado. Por ejemplo, en lugar de hacer:

```
ans = x + y;
```

podemos hacer:

```
ans = (x % mod + y % mod) % mod;
```

También es común guardar primero el módulo de cada número:

```
a %= mod;
b %= mod;
```

y luego operar:

```
suma = (a + b) % mod;
producto = (a * b) % mod;
```

Cuando hay muchas operaciones, lo importante es no esperar hasta el final si el número puede crecer demasiado. Conviene tomar módulo después de cada suma o multiplicación importante. Por ejemplo:

```
uint64_t mod = 100000007;
uint64_t ans = 0;
for(int i = 0; i < n; ++i) {
    ans = (ans + arr[i]) % mod;
}
```

La división módulo no se maneja igual que una división normal, por eso normalmente se estudia aparte.

9.2. Criba de Eratóstenes

Sirve para calcular qué números de 0 a n son primos. El arreglo `es_primo[i]` queda en `true` si i es primo. Complejidad: $O(n \log(\log(n)))$.

```
#include <bits/stdc++.h>
using namespace std;

vector<bool> criba(uint64_t n) {
    vector<bool> es_primo(n + 1, true);
    if(n >= 0) es_primo[0] = false;
    if(n >= 1) es_primo[1] = false;
    for(uint64_t i = 2; i <= n / i; ++i) {
        if(es_primo[i]) {
            for(uint64_t j = i * i; j <= n; j += i) {
                es_primo[j] = false;
            }
        }
    }
    return es_primo;
}
```

Cómo usarlo:

```
uint64_t n = 100;
vector<bool> es_primo = criba(n);
if(es_primo[37]) {
    cout << "37 es primo\n";
}
```

9.3. Coeficiente binomial iterativo

Calcula el coeficiente binomial $\binom{n}{k}$ de manera muy eficiente. Es útil cuando el resultado todavía cabe en `uint64_t`. Complejidad: $O(k)$, usando $k = \min(k, n - k)$.

```
#include <bits/stdc++.h>
using namespace std;

uint64_t binomial(uint64_t n, uint64_t k) {
    if(k > n) {
        return 0;
    }
```

```

    }
    if(k > n - k) {
        k = n - k;
    }
    uint64_t resultado = 1;
    for(uint64_t i = 1; i <= k; ++i) {
        resultado = resultado * (n - i + 1);
        resultado = resultado / i;
    }
    return resultado;
}

```

Cómo usarlo:

```

uint64_t r = binomial(5, 2);
cout << r << "\n";    // imprime 10

```

9.4. Potencia rápida con módulo

Calcula $a^b \% m$ de manera eficiente. Complejidad: $O(\log(b))$.

```

#include <bits/stdc++.h>
using namespace std;

uint64_t potencia_mod(uint64_t base, uint64_t exponente, uint64_t mod) {
    uint64_t resultado = 1;
    base %= mod;
    while(exponente > 0) {
        if(exponente % 2 == 1) {
            resultado = (resultado * base) % mod;
        }
        base = (base * base) % mod;
        exponente /= 2;
    }
    return resultado;
}

```

Cómo usarlo:

```

uint64_t MOD = 1000000007;
uint64_t r = potencia_mod(2, 10, MOD);
cout << r << "\n";    // imprime 1024

```

9.5. Binomial iterativo con módulo

Calcula $\binom{n}{k} \% m$ usando inversos modulares. Esta versión funciona cuando m es primo. Complejidad: $O(k \log(m))$, usando $k = \min(k, m - k)$.

```

#include <bits/stdc++.h>
using namespace std;

uint64_t potencia_mod(uint64_t base, uint64_t exponente, uint64_t mod) {
    uint64_t resultado = 1;
    base %= mod;
    while(exponente > 0) {
        if(exponente % 2 == 1) {
            resultado = (resultado * base) % mod;
        }
        base = (base * base) % mod;
    }

```



```

        exponente /= 2;
    }
    return resultado;
}

uint64_t inverso_mod(uint64_t x, uint64_t mod) {
    return potencia_mod(x, mod - 2, mod);
}

uint64_t binomial_mod(uint64_t n, uint64_t k, uint64_t mod) {
    if(k > n) {
        return 0;
    }
    if(k > n - k) {
        k = n - k;
    }
    uint64_t resultado = 1;
    for(uint64_t i = 1; i <= k; ++i) {
        resultado = (resultado * ((n - i + 1) % mod)) % mod;
        resultado = (resultado * inverso_mod(i, mod)) % mod;
    }
    return resultado;
}

```

Cómo usarlo:

```

uint64_t MOD = 100000007;
uint64_t r = binomial_mod(5, 2, MOD);
cout << r << "\n";    // imprime 10

```

9.6. Factorización básica

Descompone un número n como producto de factores primos. Regresa parejas de la forma primo, exponente. Complejidad: $O(\sqrt{n})$.

```

#include <bits/stdc++.h>
using namespace std;

vector<pair<uint64_t, uint64_t>> factorizar(uint64_t n) {
    vector<pair<uint64_t, uint64_t>> factores;
    for(uint64_t p = 2; p <= n / p; ++p) {
        if(n % p == 0) {
            uint64_t exponente = 0;
            while(n % p == 0) {
                n /= p;
                exponente++;
            }
            factores.push_back({p, exponente});
        }
    }
    if(n > 1) {
        factores.push_back({n, 1});
    }
    return factores;
}

```

Cómo usarlo, calculando los factores de $n = 360$:

```

uint64_t n = 360;
vector<pair<uint64_t, uint64_t>> factores = factorizar(n);

```

```
for(int i = 0; i < factores.size(); ++i) {
    cout << factores[i].first << " " << factores[i].second << "\n";
}
```

Salida, las parejas primo, exponente de la factorización $360 = 2^3 \cdot 3^2 \cdot 5^1$:

```
2 3
3 2
5 1
```

9.7. Máximo común divisor y mínimo común múltiplo

En la biblioteca `numeric` desde C++17 están disponibles las funciones de máximo común divisor `gcd` y mínimo común múltiplo `lcm`.

```
uint64_t a = 24;
uint64_t b = 36;
uint64_t mcd = gcd(a, b);
uint64_t mcm = lcm(a, b);
```

10. Estructuras de datos avanzadas

10.1. Árbol de segmentos

Un árbol de segmentos se construye a partir de un arreglo y un monoide. Permite consultar el resultado de la operación del monoide sobre cualquier intervalo del arreglo y actualizar sus elementos eficientemente.

Cada nodo representa un intervalo y almacena el resultado de combinar sus elementos. Construcción: $O(n)$. Consulta: $O(\log n)$. Actualización: $O(\log n)$. Memoria: $O(n)$.

Un monoide es una dupla $(M, \circ)$, donde M es un conjunto y $\circ$ es una operación interna que cumple las siguientes propiedades:

- Es **cerrada**: si $a, b \in M$, entonces $a \circ b \in M$.
- Es **asociativa**: si $a, b, c \in M$, entonces $(a \circ b) \circ c = a \circ (b \circ c)$.
- Tiene un **elemento neutro** $\hat{0} \in M$ tal que, $\forall a \in M: a \circ \hat{0} = a$.

Algunos ejemplos de monoides se pueden ver en la siguiente tabla

Tipo de dato	Operación	Neutro
int/float	Suma: +	0
int/float	Multipliación: ·	1
int/float	Máximo: máx	$-\infty$
int/float	Mínimo: mín	$+\infty$
int/bitset	XOR: $\oplus$	0
string	Concatenación: +	""

Una implementación simple, pero lo más general posible, se muestra a continuación.

```
template <typename T>
struct SegmentTree {
    int n;
    T neutro;
    vector<T> seg;
    function<T(T, T)> operacion;
    SegmentTree(vector<T>& a, T neutro, function<T(T, T)> operacion) {
        this->n = a.size();
        this->neutro = neutro;
    }
};
```

```

        this->operacion = operacion;
        seg.assign(4 * n, neutro);
        build(a, 1, 0, n - 1);
    }
    void build(vector<T>& a, int nodo, int izq, int der) {
        if (izq == der) {
            seg[nodo] = a[izq];
            return;
        }
        int mid = (izq + der) / 2;
        build(a, 2 * nodo, izq, mid);
        build(a, 2 * nodo + 1, mid + 1, der);
        seg[nodo] = operacion(seg[2 * nodo], seg[2 * nodo + 1]);
    }
    T query(int nodo, int izq, int der, int l, int r) {
        if (r < izq || der < l) {
            return neutro;
        }
        if (l <= izq && der <= r) {
            return seg[nodo];
        }
        int mid = (izq + der) / 2;
        T ansIzq = query(2 * nodo, izq, mid, l, r);
        T ansDer = query(2 * nodo + 1, mid + 1, der, l, r);
        return operacion(ansIzq, ansDer);
    }
    void update(int nodo, int izq, int der, int pos, T valor) {
        if (izq == der) {
            seg[nodo] = valor;
            return;
        }
        int mid = (izq + der) / 2;
        if (pos <= mid) {
            update(2 * nodo, izq, mid, pos, valor);
        } else {
            update(2 * nodo + 1, mid + 1, der, pos, valor);
        }
        seg[nodo] = operacion(seg[2 * nodo], seg[2 * nodo + 1]);
    }
    T query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
    void update(int pos, T valor) {
        update(1, 0, n - 1, pos, valor);
    }
};

```

Para construir un árbol de segmentos se debe indicar el tipo de dato de los elementos mediante el parámetro de plantilla T. Además, el constructor recibe el arreglo original, el elemento neutro del monoide y la operación que se utilizará para combinar los elementos.

Por ejemplo, para construir un árbol de segmentos sobre el monoide de los enteros con la suma usual, primero se define la operación. Esta puede declararse como una función global o mediante una función lambda dentro de main.

```

int op(int x, int y) {
    return x + y;
}

```

Como el elemento neutro de la suma es 0, el árbol se declara de la siguiente manera:

```
vector<int> arr = {5, 2, 7, 1, 3};
int neutro = 0;
SegmentTree<int> st(arr, neutro, op);
```

En este caso, `SegmentTree<int>` indica que los elementos almacenados en el árbol son de tipo `int`. Los argumentos del constructor son, respectivamente, el arreglo, el elemento neutro y la operación del monoide.

Para consultar el resultado de la operación sobre un intervalo $[i, j]$ se utiliza la función `query`. Los índices comienzan en 0 y ambos extremos del intervalo están incluidos.

```
cout << st.query(1, 4)    // 13
```

La consulta anterior calcula $2 + 7 + 1 + 3 = 13$.

Para sustituir el valor almacenado en una posición se utiliza la función `update`. El primer argumento es la posición que se desea modificar y el segundo es su nuevo valor.

```
st.update(1, 5)    // Ahora el arreglo vale {5, 5, 7, 1, 3}
cout << st.query(1, 4)    // 16
```

Después de la actualización, el valor de la posición 1 cambia de 2 a 5. Por lo tanto, la nueva consulta calcula $5 + 7 + 1 + 3 = 16$.

10.2. Conjuntos disjuntos (DSU)

La estructura de conjuntos disjuntos, conocida como **DSU** (*Disjoint Set Union*) o *Union-Find*, permite mantener una partición de varios elementos en conjuntos disjuntos. Sus principales operaciones son unir los conjuntos de dos elementos y determinar si dos elementos pertenecen al mismo conjunto.

Inicialmente, cada elemento pertenece a su propio conjunto. Por ejemplo, si tenemos 5 elementos, comenzamos con

$$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}.$$

Si unimos 0 con 1 y posteriormente 1 con 2, obtenemos

$$\{0, 1, 2\}, \{3\}, \{4\}.$$

Una implementación eficiente utiliza un representante para cada conjunto. Además, se utilizan compresión de caminos y unión por tamaño para mantener pequeños los árboles que representan los conjuntos.

```
struct DSU {
    vector<int> padre;
    vector<int> cantidad;
    int componentes;
    DSU(int n) {
        padre.resize(n);
        cantidad.assign(n, 1);
        componentes = n;

        for (int i = 0; i < n; ++i) {
            padre[i] = i;
        }
    }
    int representante(int x) {
        if (padre[x] == x) {
            return x;
        }
        return padre[x] = representante(padre[x]);
    }
    void unir(int a, int b) {
        a = representante(a);
        b = representante(b);
```

```

    if (a == b) {
        return;
    }
    if (cantidad[a] < cantidad[b]) {
        swap(a, b);
    }
    padre[b] = a;
    cantidad[a] += cantidad[b];
    --componentes;
}
bool conectados(int a, int b) {
    return representante(a) == representante(b);
}
int tam(int x) {
    return cantidad[representante(x)];
}
};

```

Para utilizar la estructura, únicamente se debe indicar el número de elementos:

```

DSU dsu(6);

dsu.unir(0, 1);
dsu.unir(1, 2);
dsu.unir(3, 4);

```

Después de estas operaciones, los conjuntos son

$\{0, 1, 2\}, \{3, 4\}, \{5\}$.

Podemos consultar si dos elementos pertenecen al mismo conjunto mediante `conectados`:

```

cout << dsu.conectados(0, 2);    // 1
cout << dsu.conectados(0, 4);    // 0

```

También podemos obtener el número de elementos del conjunto al que pertenece un elemento y la cantidad actual de conjuntos:

```

cout << dsu.tam(1);              // 3
cout << dsu.componentes;         // 3

```

En problemas de gráficas, una operación `unir(u, v)` puede utilizarse para indicar que dos vértices pertenecen a la misma componente. De esta manera, `conectados(u, v)` permite determinar si ambos vértices se encuentran en la misma componente sin necesidad de volver a recorrer la gráfica.

La inicialización de la estructura tiene complejidad $O(n)$. Utilizando compresión de caminos y unión por tamaño, las operaciones de búsqueda y unión tienen complejidad amortizada $O(\alpha(n))$, donde α es la función inversa de Ackermann, que para los tamaños utilizados en la práctica puede considerarse prácticamente constante. La memoria utilizada es $O(n)$.

11. Algunas consideraciones extras

11.1. Uso de auto

En C++, la palabra `auto` permite que el compilador deduzca automáticamente el tipo de una variable. Esto es muy útil cuando el tipo de dato es **muy largo**, **difícil de escribir** o simplemente **incómodo de leer**. Por ejemplo, en lugar de escribir:

```
vector<int>::iterator it = v.begin();
```

podemos escribir:

```
auto it = v.begin();
```

y el compilador deduce automáticamente el tipo correcto. Esto se usa mucho en programación competitiva cuando trabajamos con:

- iteradores
- set
- map
- vector
- funciones de la biblioteca estándar que regresan tipos largos

Por ejemplo:

```
set<int> s;  
auto it = s.find(7);
```

Aquí, `s.find(7)` regresa un iterador de `set<int>`, pero no hace falta escribir todo el tipo. También es común en `map`:

```
map<string, int> m;  
auto it = m.begin();
```

o en recorridos:

```
for(auto it = v.begin(); it != v.end(); ++it) {  
    cout << *it << " ";  
}
```

y también:

```
for(auto it = m.begin(); it != m.end(); ++it) {  
    cout << (*it).first << " " << (*it).second << "\n";  
}
```

La ventaja principal de `auto` es que hace el código **más corto** y muchas veces **más claro**, sobre todo cuando el tipo completo no aporta mucho y solo estorba visualmente.

11.2. ¿Qué hacer cuando voy a resolver un problema?

Del prólogo de las notas del Club de Algoritmos UAM-Azcapotzalco, escritas por el Dr. Rodrigo Alexander Castro Campos, considera lo siguiente al resolver un problema:

Antes de enviar:

- Escribe algunos casos de prueba simples, si los ejemplos no son suficientes.
- ¿Están cerca los límites de tiempo? Si es así, genera casos máximos.
- ¿El uso de memoria está dentro de los límites?
- ¿Podría haber algún desbordamiento?
- Asegúrate de enviar el archivo correcto.

Respuesta incorrecta:

- ¡Imprime tu solución! Imprime también la salida de depuración.

- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?
- ¿Puede tu algoritmo manejar todo el rango de entrada?
- Lee nuevamente el enunciado completo del problema.
- ¿Manejas todos los casos límite correctamente?
- ¿Has entendido correctamente el problema?
- ¿Hay variables no inicializadas?
- ¿Algún desbordamiento?
- ¿Confundes N y M , 1 y l , etc.?

- ¿Estás seguro de que tu algoritmo funciona?
- ¿Qué casos especiales no has considerado?
- ¿Estás seguro de que las funciones STL que usas funcionan como piensas?
- Añade algunas aserciones, tal vez vuelve a enviar.
- Crea algunos casos de prueba para probar tu algoritmo.
- Revisa el algoritmo con un caso simple.
- Revisa esta lista nuevamente.
- Explica tu algoritmo a un compañero de equipo.

Error de tiempo de ejecución:

- ¿Has probado todos los casos límite localmente?
- ¿Hay variables no inicializadas?
- ¿Estás leyendo o escribiendo fuera del rango de algún vector?
- ¿Alguna aserción que podría fallar?
- ¿Alguna posible división por 0? Por ejemplo, `mod 0`.

- ¿Alguna posible recursión infinita?
- ¿Punteros o iteradores inválidos?
- ¿Estás usando demasiada memoria?
- Depura con reenvíos. Por ejemplo, señales remapeadas, ver Varios.

Límite de tiempo excedido:

- ¿Tienes algún ciclo infinito posible?
- ¿Cuál es la complejidad de tu algoritmo?
- ¿Estás copiando muchos datos innecesarios? Usa referencias.
- ¿Qué tan grandes son la entrada y salida? Considera `scanf`.
- ¿Qué piensan tus compañeros sobre tu algoritmo?

Límite de memoria excedido:

- ¿Cuál es la cantidad máxima de memoria que debería necesitar tu algoritmo?
- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?

Fuente: <https://github.com/darkmx/uam-icpc/>